

CHITTAGONG UNIVERSITY OF ENGINEERING AND TECHNOLOGY



Department of Electronics and Telecommunication Engineering

PROJECT REPORT

Smart Energy Monitoring Fault-Aware Load Control Using IoT

Course No: ETE 416

Course Title: Internet of Things (IoT) Systems Design

Submitted By		Submitted To	
Name:	Ishrat Jahan Mily	Taieba Taher	
ID:	2008013	Assistant Professor	
Name:	Maitri Chowdhury Achal	Dept. of ETE, CUET	
ID:	2008035		
Name:	Tasfia Islam		
ID:	2008053		

Contents

1	Introduction	3
2	Motivations	3
3	Objectives	4
4	System Architecture	4
4.1	Hardware Architecture	4
4.2	Software Architecture	5
4.3	Cost Analysis	6
5	Methodology	6
5.1	Overall Workflow	6
5.2	Hardware Implementation	8
5.3	System Initialization and Connectivity	9
5.4	Sensor Data Acquisition	9
5.5	Electrical Parameter Calculation	9
5.6	Cloud-Based Data Transmission	10
5.7	Firebase Monitoring Dashboard	10
5.8	Remote Load Control Mechanism	11
5.9	Overload Detection and Protection	11
5.10	Heartbeat and System Status Monitoring	11
5.11	Task-Based Execution Using FreeRTOS	11
5.12	Continuous System Operation	12
6	Results	12
6.1	Hardware Implementation Result	13
6.2	Firebase Realtime Database Result	14
6.3	Dashboard Monitoring Result	15
6.4	Remote Load Control Result	16
6.5	Overload Protection Result	17
6.6	Overall System Performance	17
7	Limitations	18
8	Conclusion	19
9	Program Code	19

10 Project Resources	22
10.1 Source Code Repository	22

1 Introduction

This project presents a Smart Energy Monitoring and Load Control System using ESP32, Firebase, and a web-based dashboard for real-time monitoring and control of three independent AC loads. In many homes, labs, and small industrial setups, the electrical condition of common AC loads is not visible in real time, which often causes energy waste, unsafe manual monitoring, and late detection of overload or abnormal current conditions. To solve this problem, the proposed system measures important electrical parameters such as voltage, current, power, and energy consumption using sensing modules connected to the ESP32. The collected data is processed and uploaded to Firebase Realtime Database, from where it is displayed live on an HTML dashboard. The dashboard also allows the user to remotely control each load independently through relay switching. A key feature of the system is its intelligent protection mechanism. If any load experiences overcurrent, overload, or abnormal operating condition, the system automatically detects it, isolates the affected load, and displays an emergency alert on the dashboard. In addition, the system includes real-time cloud connectivity, online/offline device status detection, and summary monitoring, making it more efficient, reliable, and user-friendly than conventional monitoring methods.

Overall, this project provides a complete IoT-based smart solution for energy monitoring, fault detection, safety protection, and intelligent AC load control on a single integrated platform.

2 Motivations

The motivation behind this project is to develop a smart and real-time energy monitoring system for common AC loads used in homes, laboratories, and small industrial environments. In many existing systems, there is no integrated platform to measure voltage, current, power, and energy consumption together, and important fault conditions such as overcurrent, sudden spikes, and prolonged high load are often not detected in real time.

This lack of visibility and automation leads to energy waste, unsafe manual monitoring, delayed fault detection, and inefficient load management. Moreover, many conventional solutions do not provide intelligent control, automatic response, or remote accessibility.

To overcome these limitations, this project proposes an IoT-based smart system capable of real-time monitoring, automatic fault detection, emergency protection, and independent remote load control through a cloud-connected dashboard. The goal is to create a system that is more safe, efficient, intelligent, and practical for modern energy monitoring applications.

Figure 1: Hardware architecture of the smart energy monitoring and load control system.

4.2 Software Architecture

The software architecture is developed using **Arduino IDE**, **Firebase Realtime Database**, and a **web-based dashboard built with HTML, CSS, and JavaScript**. The ESP32 firmware handles sensor data acquisition, electrical parameter calculation, overload detection, relay control, and communication with the cloud.

Firebase is used as a real-time database to store and synchronize system data between the ESP32 and the dashboard. The web dashboard provides a user-friendly interface for **real-time monitoring of voltage, current, power, energy consumption, emergency status, and load control**. This architecture ensures **real-time operation, remote accessibility, intelligent fault detection, and efficient user interaction**.

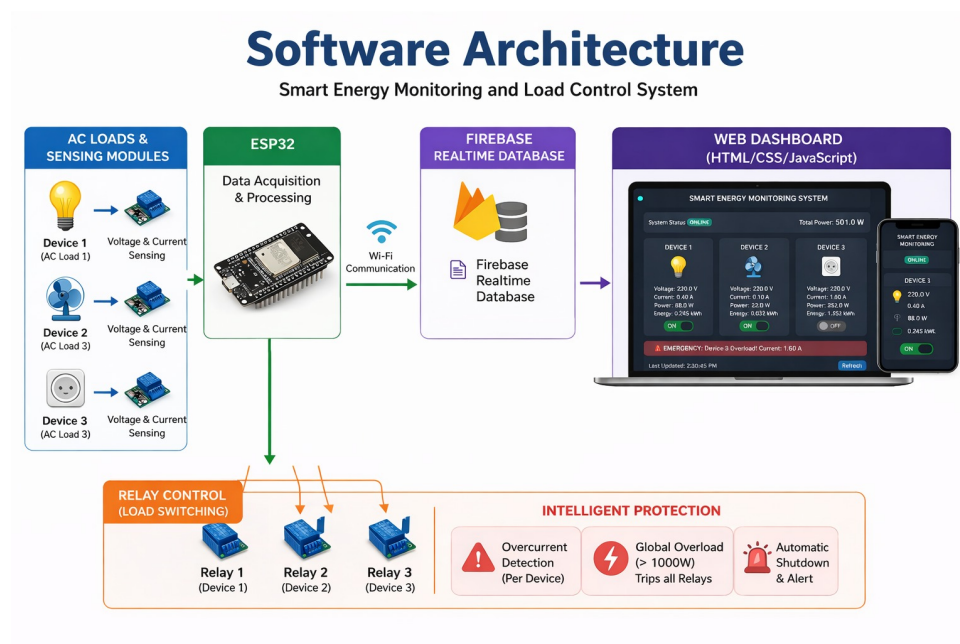


Figure 2: Software architecture showing ESP32, Firebase, and web dashboard communication.

4.3 Cost Analysis

The estimated cost of the components used in the proposed smart energy monitoring and safety system is presented in Table 1.

Table 1: Cost analysis of the proposed system

SL. No.	Component Name	Quantity	Total Cost (BDT)
1	ESP32 Microcontroller	1	460
2	ZMPT101B Voltage Sensor	2	250
3	ZMCT103C Current Sensor	3	810
4	4-Channel Relay Module	1	270
5	2-Pin Socket	2	280
6	2-Pin Power Cable	1	100
7	Bulb Holder	1	80
8	Breadboard	1	130
9	Jumper Wire	Various	70
10	AC Bulb (100W)	1	100
11	Wire	1	80
12	AC Fan	1	–
13	Battery Charging Device (Mobile Charger)	1	–
Total			2630

5 Methodology

This chapter describes the complete methodology followed to implement the proposed smart energy monitoring and safety system. The methodology includes the overall workflow of the system, initialization and communication setup, sensor data acquisition, electrical parameter calculation, cloud-based data transmission, relay control mechanism, overload protection, and continuous task-based system operation using FreeRTOS.

5.1 Overall Workflow

The overall methodology of the proposed smart energy monitoring and safety system is illustrated in Figure 3.

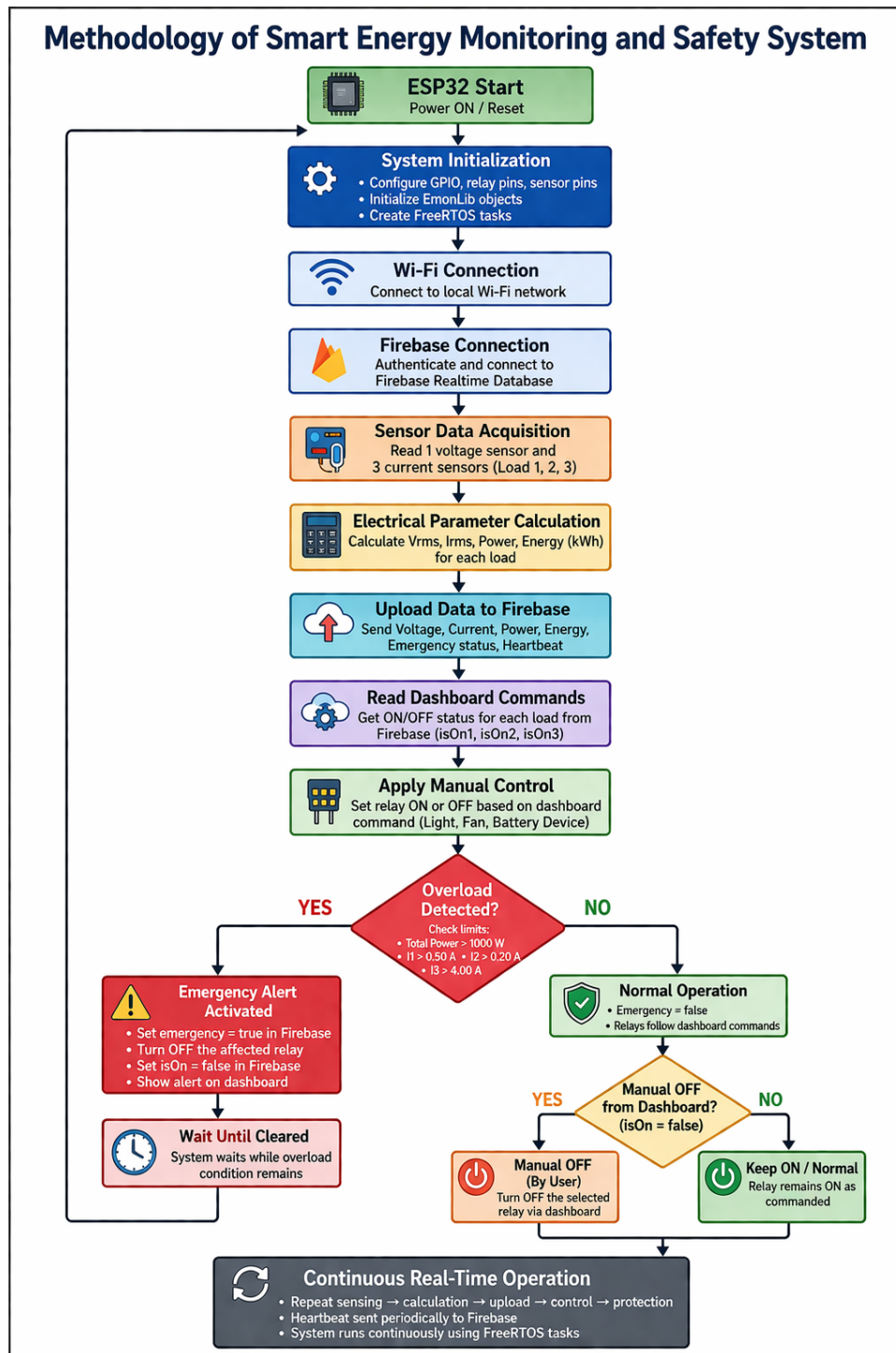


Figure 3: Overall workflow of the proposed smart energy monitoring and safety system

As shown in Figure 3, the system begins by initializing the ESP32 microcontroller and establishing communication with Wi-Fi and Firebase. After successful connection, the system continuously reads the voltage and current sensor values from the connected loads. Based on these measured values, the system calculates the essential electrical parameters such as voltage, current, power, and energy consumption. The calculated data are then uploaded to the Firebase Realtime Database for real-time monitoring through

the dashboard. Simultaneously, the system checks for remote ON/OFF control commands and updates the relay states accordingly. In addition, overload conditions are monitored continuously to ensure the safe operation of the connected loads. If any overload is detected, the corresponding relay is automatically turned OFF; otherwise, the system continues its normal operation in a continuous loop.

5.2 Hardware Implementation

The hardware implementation of the proposed system is shown in Figure 4. The system was physically constructed using an ESP32 development board, voltage sensing module, three current sensing channels, relay module, and three different electrical loads. The complete hardware setup was designed to monitor the electrical parameters of each load and provide remote control and protection features through the integrated cloud platform.

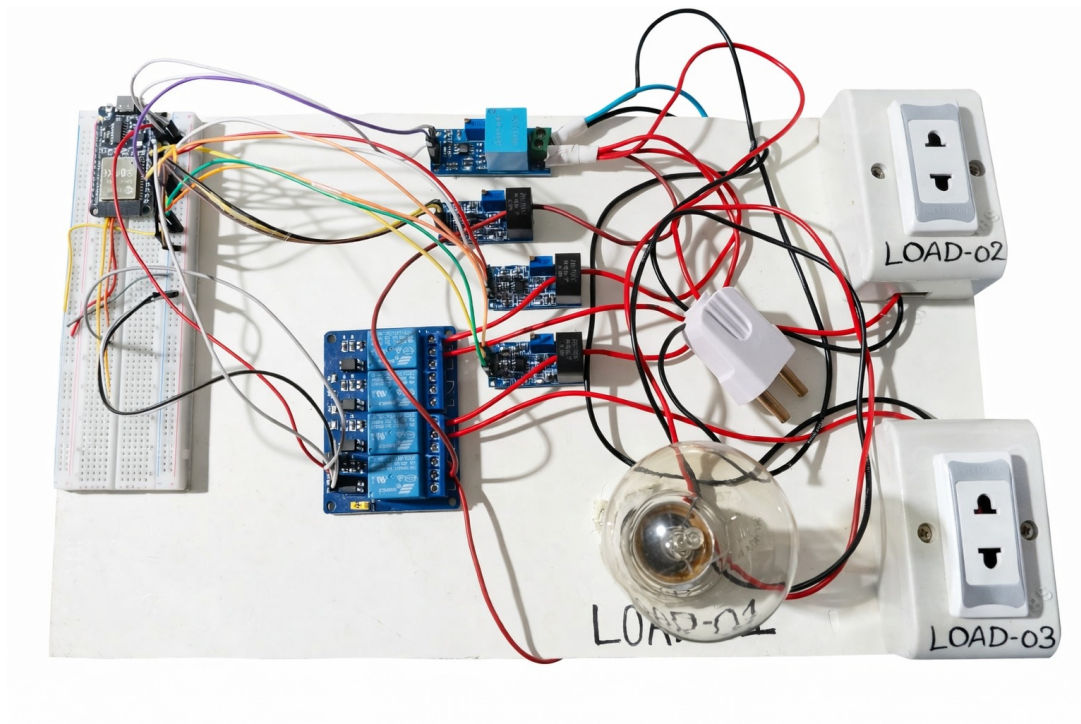


Figure 4: Hardware implementation of the smart energy monitoring and safety system

5.3 System Initialization and Connectivity

The system operation begins with the initialization of the ESP32 microcontroller, where all required GPIO pins, analog sensor inputs, and relay output pins are configured. After initialization, the ESP32 connects to the available Wi-Fi network and then establishes communication with the Firebase Realtime Database using the predefined credentials. This connection enables the system to upload sensor data and receive remote control commands from the dashboard.

5.4 Sensor Data Acquisition

The system uses one voltage sensor and three current sensors to monitor the electrical condition of three individual loads. The sensors are connected to the analog input pins of the ESP32 for continuous data acquisition. In this setup, Load 1 is a light, Load 2 is a fan, and Load 3 is a battery-powered device. Each load is monitored independently for separate analysis.

5.5 Electrical Parameter Calculation

After collecting the raw analog sensor values, the system processes the measured data using the `EmonLib` library. This library is used to calculate the major electrical parameters required for monitoring and analysis.

The calculated parameters include:

- RMS Voltage (V_{rms})
- RMS Current (I_{rms})
- Real Power (W)
- Energy Consumption (kWh)

5.6 Cloud-Based Data Transmission

After calculating the electrical parameters, the ESP32 transmits the measured data to the Firebase Realtime Database through Wi-Fi. This enables real-time remote monitoring of the system through an internet-based platform.

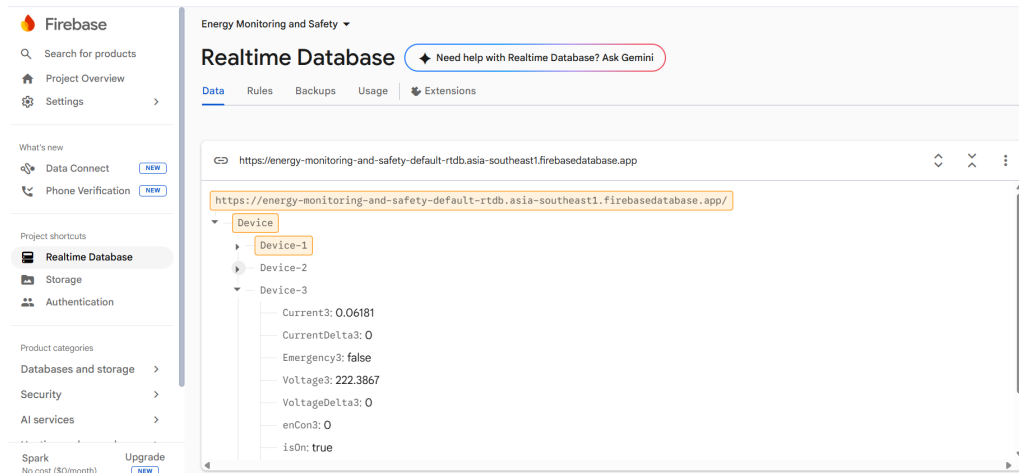


Figure 5: Firebase Realtime Database storing measured electrical parameters and device control states

5.7 Firebase Monitoring Dashboard

The measured electrical parameters uploaded to Firebase are visualized through a real-time dashboard interface. The dashboard provides users with live access to the electrical condition of the connected load. It allows users to observe the system status remotely and also provides control options for switching the connected loads ON or OFF.

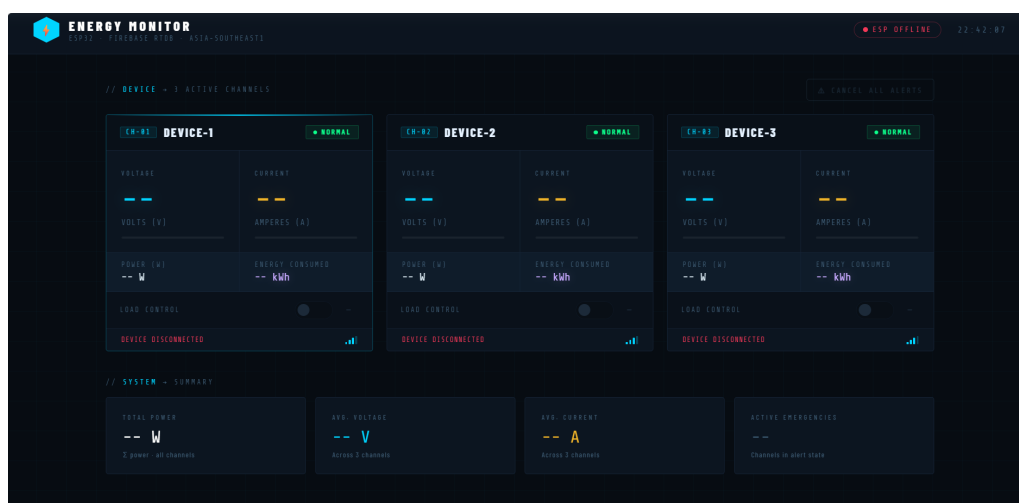


Figure 6: Firebase-based real-time monitoring dashboard of the proposed system

5.8 Remote Load Control Mechanism

In addition to real-time monitoring, the system supports remote load control through the Firebase dashboard. Each load is connected to a relay, which is controlled based on the ON/OFF status stored in Firebase (e.g., `Device/Device-1/isOn`). The ESP32 periodically reads these values and updates the corresponding relay states. This enables remote operation of the light, fan, and battery-powered device, making the system suitable for smart energy management.

5.9 Overload Detection and Protection

To ensure safe operation, the system continuously checks whether the measured current or total power exceeds the predefined threshold values. If an overload is detected, the corresponding relay is turned OFF automatically and the emergency status is updated in Firebase. This improves the safety and reliability of the system.

5.10 Heartbeat and System Status Monitoring

A heartbeat mechanism is used to indicate whether the ESP32 is active and connected to Firebase. The system periodically updates a timestamp in the Firebase path `Device/heartbeat`, which acts as a real-time status indicator for the cloud-based system.

5.11 Task-Based Execution Using FreeRTOS

To ensure smooth real-time performance, the proposed system is implemented using FreeRTOS on the dual-core ESP32 platform. Instead of running all operations sequentially inside a single loop, the system functionality is divided into multiple parallel tasks.

The task-based implementation used in this project includes:

- **Core 0 Task:** Responsible for Wi-Fi connection, Firebase authentication, and heartbeat transmission.
- **Core 1 Task:** Responsible for reading sensor values, calculating electrical parameters, checking overload conditions, and uploading data to Firebase.
- **Relay Poll Task:** Responsible for continuously checking relay control commands from Firebase and updating relay outputs accordingly.

This task-based structure improves the responsiveness of the system and ensures that sensing, communication, monitoring, and relay control can all run efficiently in parallel.

5.12 Continuous System Operation

After completing initialization and task allocation, the system enters continuous real-time operation. The ESP32 repeatedly performs the following actions:

- Reads voltage and current values
- Calculates electrical parameters
- Uploads data to Firebase
- Checks remote ON/OFF control commands
- Controls relay outputs
- Monitors overload conditions
- Updates heartbeat signal

6 Results

This chapter presents the experimental results and performance evaluation of the proposed smart energy monitoring and safety system. The system was tested in terms of hardware implementation, real-time data monitoring, cloud data transmission, remote load control, overload protection, and overall operational reliability.

6.1 Hardware Implementation Result

The final hardware setup of the proposed system is shown in Figure 7. The system was successfully implemented using the ESP32 development board, voltage sensor, current sensors, relay module, and connected electrical loads. The hardware operated properly and performed sensing, relay control, and cloud communication successfully.

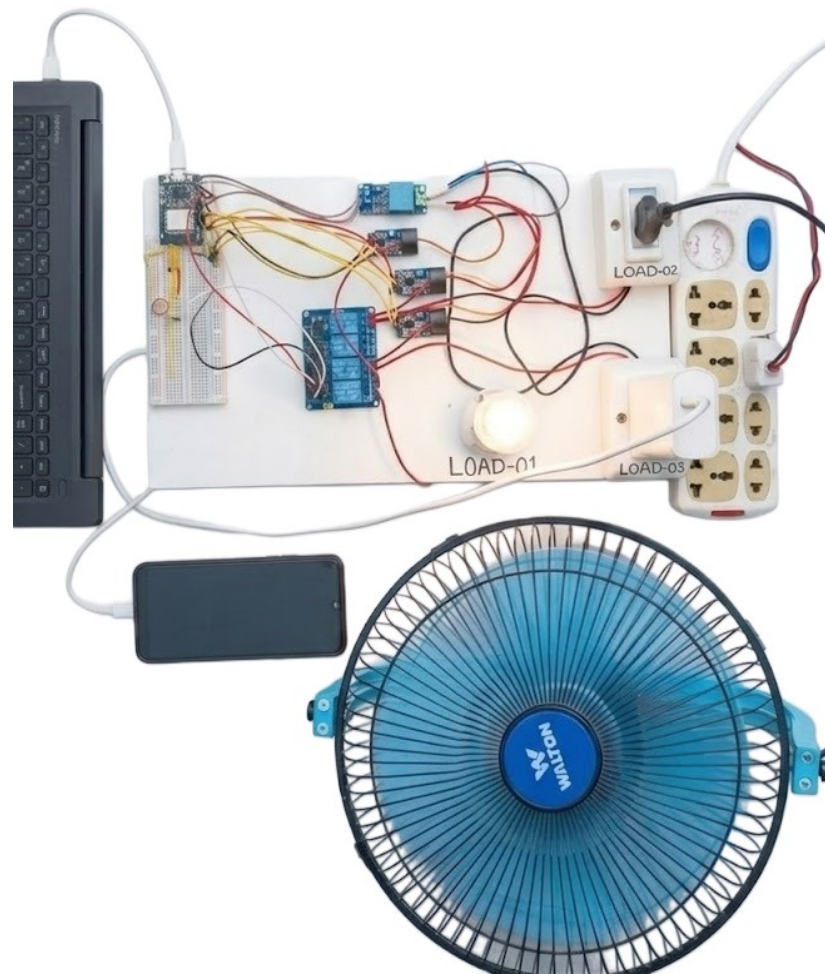


Figure 7: Final hardware implementation of the proposed smart energy monitoring and safety system

6.2 Firebase Realtime Database Result

The measured electrical parameters and control states of all three loads were successfully updated in the Firebase Realtime Database. Figure 8 shows the realtime database entries for Device 1, Device 2, and Device 3. The continuous update of voltage, current, relay state, and emergency status confirms that the cloud-based monitoring system operated successfully.

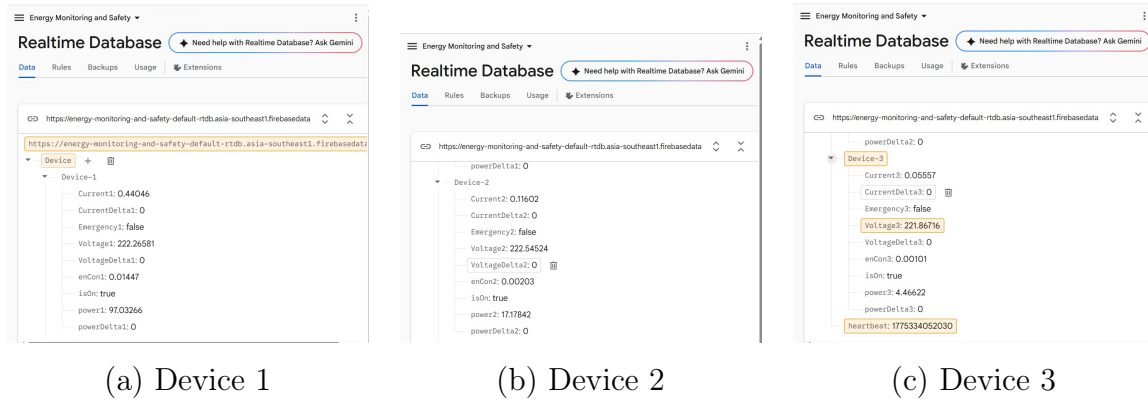
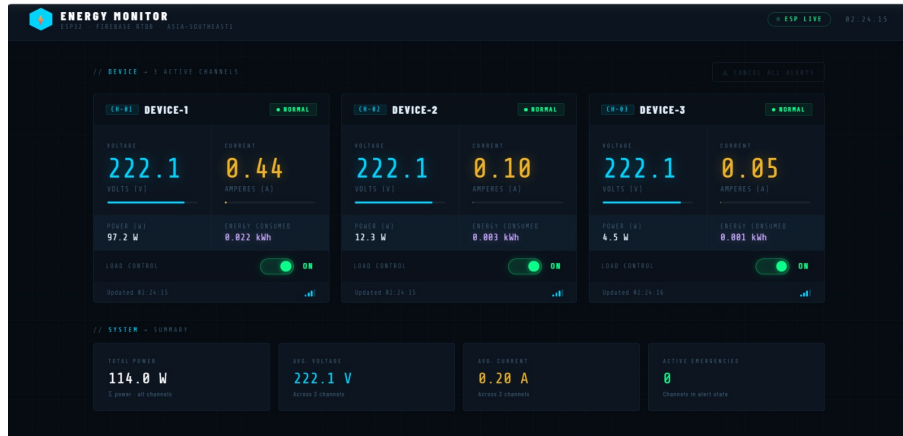


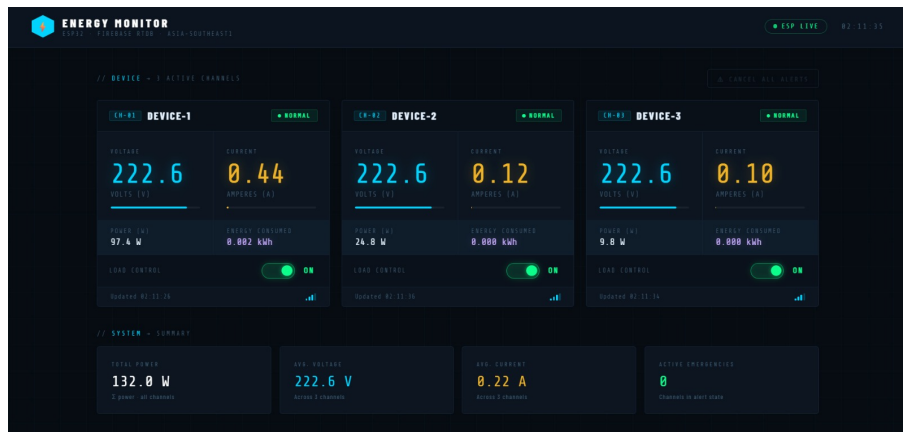
Figure 8: Firebase Realtime Database entries for the three connected devices

6.3 Dashboard Monitoring Result

The real-time dashboard interface successfully displayed the uploaded electrical parameters of the connected loads. Figure 9 shows the dashboard readings before and after a change in the operating condition of Device 2 (fan).



(a) Initial dashboard reading



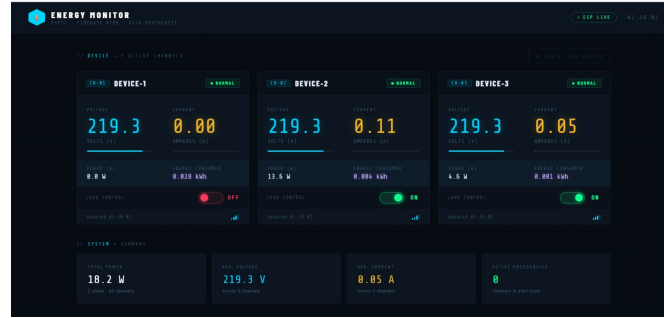
(b) Updated dashboard reading after Device 2 current change

Figure 9: Realtime monitoring dashboard showing live parameter updates

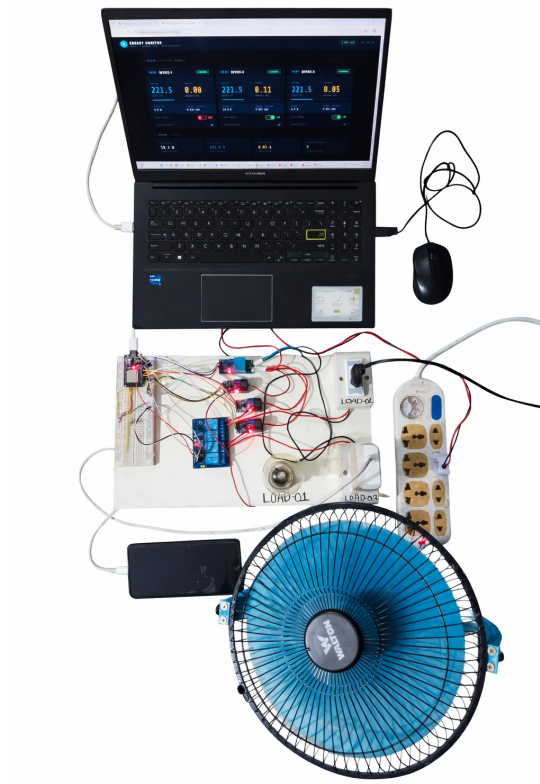
It was observed that the dashboard values were updated successfully in real time according to the load condition. This confirms the effectiveness of the proposed system as an IoT-based realtime monitoring platform.

6.4 Remote Load Control Result

The remote load control feature of the proposed system was tested successfully. When the load was turned OFF from the dashboard, the ESP32 updated the corresponding relay state and disconnected the load accordingly.



(a) Load OFF command from dashboard



(b) Actual load turned OFF in hardware

Figure 10: Remote load control result showing dashboard command and corresponding hardware response

This confirms that the proposed system can control connected loads remotely through the dashboard.

6.5 Overload Protection Result

The overload protection mechanism of the proposed system was also tested successfully. During testing, when the measured current or total power exceeded the predefined threshold value, the corresponding relay was automatically turned OFF and the emergency status was updated in Firebase.

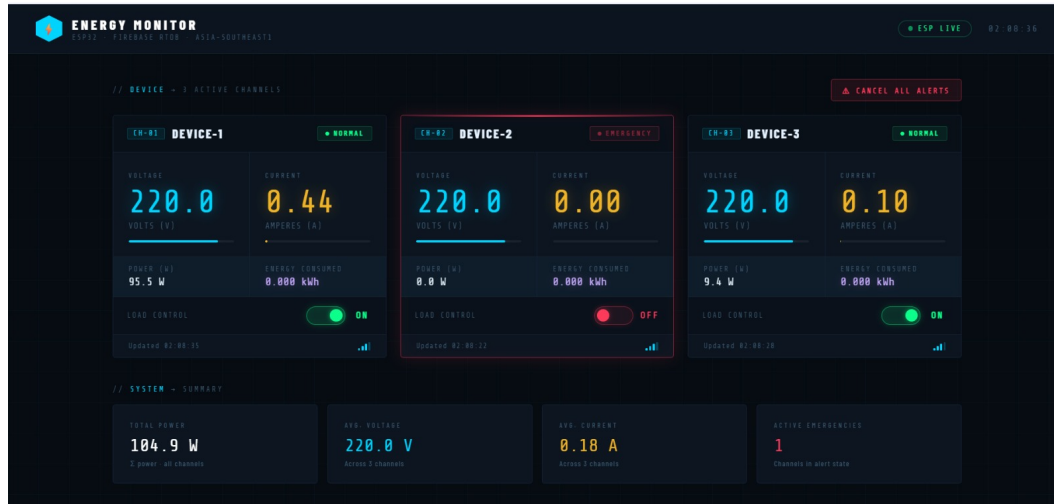


Figure 11: System response during overload condition showing emergency status and automatic relay shutdown

This result confirms that the system was able to detect abnormal electrical conditions and respond automatically to protect the connected load. Therefore, the overload protection feature significantly improved the safety and reliability of the proposed system.

6.6 Overall System Performance

Based on the practical implementation and testing results, the proposed smart energy monitoring and safety system successfully performed all of its intended functions. The system was able to:

- measure electrical parameters in real time,
- upload data to Firebase successfully,
- provide remote monitoring through the dashboard,
- control connected loads remotely,
- detect overload conditions automatically, and
- disconnect the affected load for safety.

Therefore, the overall performance of the system can be considered successful and suitable for real-time smart energy monitoring, control, and protection applications.

Table 2: Summary of System Performance Evaluation

Tested Function	Result
Voltage Measurement	Successful
Current Measurement	Successful
Power Calculation	Successful
Energy Monitoring	Successful
Firebase Data Upload	Successful
Realtime Monitoring	Successful
Remote Relay Control	Successful
Overload Detection	Successful
Automatic Relay Shutdown	Successful
Heartbeat Status Update	Successful

The results demonstrate that the proposed system achieved its intended objective of developing a smart energy monitoring and safety platform with real-time monitoring, remote control, and overload protection capability.

7 Limitations

Although the proposed smart energy monitoring and safety system was implemented successfully, it has several limitations. First, the system currently depends on proper sensor calibration and stable internet connectivity for accurate monitoring and cloud communication. Any sensor drift or network interruption may affect the realtime performance of the system.

Second, the current implementation is limited to a prototype-level design and has been tested with a small number of loads. Therefore, large-scale industrial deployment would require further hardware robustness, electrical isolation, and system scalability.

In addition, machine learning-based fault prediction or intelligent anomaly detection could not be incorporated in this work, as a sufficient real-time historical database was not available during the implementation phase. Since no large and structured dataset was generated from the system over a long operating period, advanced data-driven predictive analysis could not be applied.

Furthermore, the system currently focuses mainly on electrical parameter monitoring, relay-based control, and overload protection. Additional industrial monitoring features such as predictive maintenance, advanced analytics, and automated fault classification remain beyond the scope of the present work.

8 Conclusion

This project presented the design and implementation of a smart energy monitoring and safety system based on the ESP32 microcontroller and Firebase cloud platform. The developed system was able to monitor voltage, current, power, and energy consumption of multiple electrical loads in real time and provide remote monitoring through a cloud-based dashboard.

The proposed system also supported remote load control using relay switching and ensured safety through an automatic overload protection mechanism. As a result, the system successfully combined realtime sensing, cloud communication, remote control, and protection within a single IoT-based platform.

Overall, the implementation and testing results demonstrated that the proposed system is a reliable, low-cost, and effective solution for smart energy monitoring and safety applications. The developed system can serve as a useful prototype for future expansion toward more advanced industrial monitoring and intelligent energy management systems.

9 Program Code

The complete implementation code of the proposed smart energy monitoring and safety system is presented below.

```
1 WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
2
3 while (WiFi.status() != WL_CONNECTED) {
4     delay(300);
5 }
6
7 config.api_key      = API_KEY;
8 config.database_url = DATABASE_URL;
9
10 Firebase.begin(&config, &auth);
11 Firebase.reconnectWiFi(true);
```

Listing 1: Wi-Fi and Firebase Initialization

```
1 emon1.calcVI(40, 2000);
2 emon2.calcVI(40, 2000);
3 emon3.calcVI(40, 2000);
4
5 float Vrms = emon1.Vrms / 4.0;
6 float Irms1 = emon1.Irms / 4.0;
7 float P1    = emon1.realPower / 16.0;
8
9 float Irms2 = emon2.Irms / 4.0;
10 float P2    = emon2.realPower / 16.0;
11
12 float Irms3 = emon3.Irms / 4.0;
13 float P3    = emon3.realPower / 16.0;
```

Listing 2: Sensor Data Acquisition and Electrical Parameter Calculation

```
1 Firebase.RTDB.setFloat(&fbdo1, "Device/Device-1/Voltage1", Vrms);
2 Firebase.RTDB.setFloat(&fbdo1, "Device/Device-1/Current1", Irms1);
3 Firebase.RTDB.setFloat(&fbdo1, "Device/Device-1/power1", P1);
4
5 Firebase.RTDB.setFloat(&fbdo1, "Device/Device-2/Current2", Irms2);
6 Firebase.RTDB.setFloat(&fbdo1, "Device/Device-3/Current3", Irms3);
```

Listing 3: Uploading Data to Firebase

```
1 if (Firebase.RTDB.getBool(&fbdo2, "Device/Device-1/isOn")) {
2     bool state = fbdo2.boolData();
3     digitalWrite(RELAY_1_PIN, state ? HIGH : LOW);
4 }
```

Listing 4: Remote Load Control from Firebase

```
1 bool overload1 = (Irms1 > CURRENT_LIMIT_1);
2 bool overload2 = (Irms2 > CURRENT_LIMIT_2);
3 bool overload3 = (Irms3 > CURRENT_LIMIT_3);
4
5 if (overload1) {
6     digitalWrite(RELAY_1_PIN, LOW);
7     Firebase.RTDB.setBool(&fbdo1, "Device/Device-1/Emergency1", true);
8 }
```

Listing 5: Overload Detection and Protection

```
1 xTaskCreatePinnedToCore(Task_Core0, "WiFi", 8192, NULL, 2, NULL, 0);
2 xTaskCreatePinnedToCore(Task_Core1, "Sensor", 16384, NULL, 1, NULL, 1);
3 xTaskCreatePinnedToCore(Task_RelayPoll, "Relay", 8192, NULL, 1, NULL, 1);
```

Listing 6: FreeRTOS Task Creation

10 Project Resources

10.1 Source Code Repository

GitHub Repository: Smart Energy Monitoring Fault-Aware Load Control Using IoT
GitHub Repository: ESP32 code

References

- [1] Espressif Systems, “ESP32 Series Datasheet,” Available: <https://www.espressif.com/>
- [2] Google, “Firebase Realtime Database Documentation,” Available: <https://firebase.google.com/docs/database>
- [3] OpenEnergyMonitor, “EmonLib Energy Monitoring Library,” Available: <https://github.com/openenergymonitor/EmonLib>
- [4] K. Ashton, “That ‘Internet of Things’ Thing,” RFID Journal, 2009.
- [5] L. Atzori, A. Iera, and G. Morabito, “The Internet of Things: A survey,” Computer Networks, vol. 54, no. 15, pp. 2787–2805, 2010.
- [6] A. A. Khan, M. H. Rehmani, and A. Rachedi, “Cognitive-radio-based Internet of Things: Applications, architectures, spectrum related functionalities, and future research directions,” IEEE Wireless Communications, vol. 24, no. 3, pp. 17–25, 2017.
- [7] S. K. Sahoo and D. K. Sahoo, “Smart Energy Monitoring System using IoT,” International Journal of Engineering Research & Technology (IJERT), 2019.